# PROJET BASE DE DONNÉES

# Plateforme de Streaming

**Dirigé par Wiam MEJJATI,**

**Pio DUBRAIL, Amîn MEZOUER**

**DIRECTION : BEATRICE FINANCE**

**ANNÉE UNIVERSITAIRE : 2025-2026**

# PHASE 1

## Cahier Des Charges

Notre projet consiste à modéliser la base de données d'une **plateforme de streaming**, en prenant en compte l'ensemble des contraintes nécessaires à son bon fonctionnement.
L'objectif est de concevoir une structure de données, permettant de gérer efficacement les utilisateurs, abonnements, films, séries, contenus, épisodes, acteurs, personnages et réalisateurs.

Chaque utilisateur dispose d'un **compte personnel** lui permettant d'accéder aux contenus de la plateforme.
Lors de son inscription, il/elle doit renseigner les informations suivantes :

- Nom,
- Prénom,
- Adresse e-mail (unique)
- Pays de résidence
- Date de naissance
- Mot de passe (stocké de manière sécurisé et hachée)

Après la création du compte, l'utilisateur choisit un **type d'abonnement** parmi :

- Basique : Accès limité (50 contenus différents visionnables par mois, aucun téléchargement)
- Premium : Accès complet (Téléchargements et visionnages illimités)

Le type d'abonnement peut être changé uniquement lorsque le précédent expire et n'est pas renouvelé.
Une fois inscrit, il peut :

- Visionner et télécharger des contenus (films ou épisodes de séries).
- Aimer un contenu (like).
- Consulter les informations détaillées d'un contenu (titre, durée, date de sortie, etc.).

Le cœur de la base de données sera le catalogue de contenus, qui regroupera les films et les séries. Chaque contenu sera identifié par un titre, une durée, une date de sortie et une limite d'âge. Certains contenus pourront être restreints en fonction de l'âge : les utilisateurs ne remplissant pas les conditions ne pourront pas y accéder.

Les films comprendront des informations supplémentaires, telles que l'année de sortie, le pays de production et le (ou les) réalisateurs et personnages associés.
Les séries, quant à elles, seront composées de saisons et d'épisodes.

Chaque épisode possédera un titre, un numéro d'épisode, un numéro de série, une durée et une date de diffusion, et sera rattaché à une série.

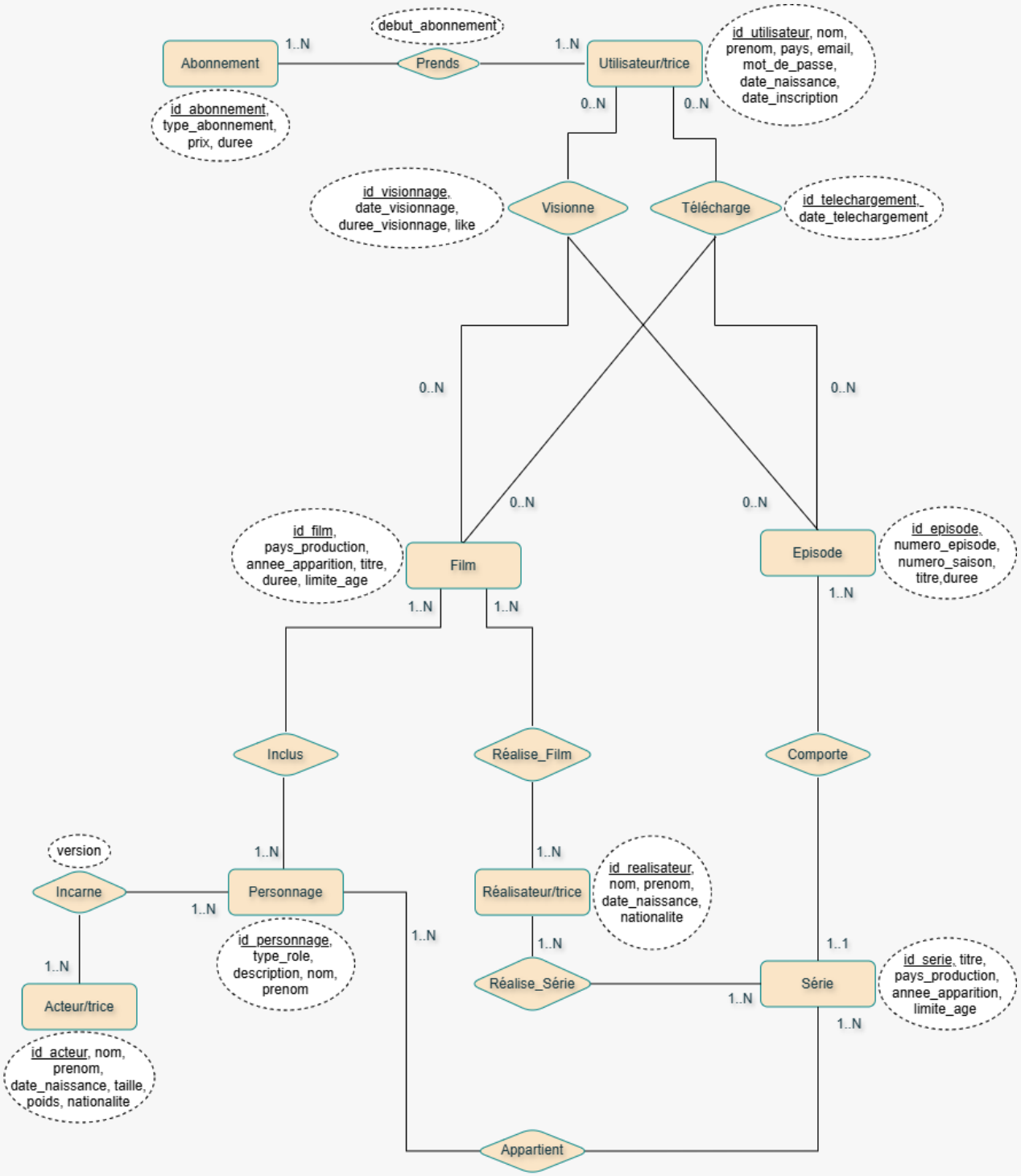
Les réalisateurs et les acteurs auront également leur place dans la base de données.
Un(e) réalisateur(trice) pourra être associé(e) à plusieurs films ou séries selon ses œuvres, et aura la possibilité de proposer de nouveaux contenus.
Ces propositions devront être validées par la plateforme avant leur mise en ligne afin de garantir la qualité et la conformité du contenu.

Les acteurs, de leur côté, pourront participer à plusieurs films et séries et incarner différents personnages.
Chaque personnage sera défini par un nom, un prénom, un rôle et une brève description, permettant d'identifier précisément quel acteur a joué dans quelle œuvre et quel rôle il a interprété.
Les informations personnelles des acteurs (nom, prénom, date de naissance, nationalité, etc.) seront enregistrées afin de permettre des recherches et des rapports statistiques.

La base de données enregistrera également les activités des utilisateurs sur la plateforme.
Lorsqu'un(e) utilisateur/trice visionnera un film ou un épisode, la date du visionnage et la durée regardée seront conservées.
Si un contenu est téléchargé, la date du téléchargement sera enregistrée.
Les likes des utilisateurs permettront de suivre la popularité des contenus et d'améliorer les recommandations personnalisées.

Cette base de données aura donc pour rôle de relier les utilisateurs, les abonnements, les contenus audiovisuels et les personnes qui y participent, tout en garantissant la sécurité des informations et la gestion des droits et des vues associées.

# Modèle Entité/Relation



# Contraintes d'Intégrités complexes

## Table : Personnage

• Un personnage peut être incarné par plusieurs acteurs uniquement s'ils représentent des versions différentes du personnage (jeune, adulte, âgé, etc.).

## Table : Episode

• La numérotation des épisodes et des saisons doit suivre une logique croissante et doit être unique au sein d'une même série (pas de doublon de numéro d'épisode dans la même saison).

• Lorsque qu'une saison est achevée, on ne peut plus ajouter de nouveaux épisodes dans cette saison, l'ajout des épisodes doit se faire dans l'ordre croissant.

## Table : Visionne

• Un(e) utilisateur(trice) peut revisionner un contenu mais dans le comptage de visionnage on comptera le contenu comme étant visionner qu'une fois par l'utilisateur/trice.

• Si un(e) utilisateur(trice) tente de visionner plus d'épisodes ou de film que sa limite mensuelle (abonnement basique), le trigger doit bloquer l'action.

• Si un(e) utilisateur(trice) tente de visionner un contenu avec une limite d'âge supérieur à son âge, le trigger doit bloquer l'action et générer.

## Table : Abonnement

• Un(e) utilisateur(trice) ne peut pas avoir deux abonnements actifs à la fois, il/elle devra attendre la fin de son abonnement pour en prendre un nouveau.

• Si un abonnement expire, l'utilisateur ne peut ni visionner ni télécharger de films ou d'épisodes.

• Si un un(e) utilisateur(trice) a un abonnement basique, il peut visionner plus de 50 épsiodes/films sur sa durée d'abonnement.

**Table : Réalisateur/trice**

• Un(e) réalisateur(trice) ne peut pas être supprimé s'il/elle est assigné(e) seul(e) à un film ou une série.

• La plateforme devra vérifier si l'ajout d'un film ou d'une série de la part d'un(e) réalisateur(trice) est conforme au règle (Exemple : titre non insultant, pas de copyright, etc.)

**Table : Acteur/trice**

• Un(e) acteur(trice) ne peut pas incarner deux personnages différents dans une même série ou film.

**Table : Télécharge**

• Seuls les utilisateurs qui ont un abonnement Premium peuvent faire des téléchargements de contenu (films ou épisode de série).

• La limite de contenu téléchargé est de 20, donc si la limite est atteinte, lorsque qu'un contenu est téléchargé la plus ancienne sera supprimé par soucis de stockage.

• Un même contenu ne peut pas apparaitre 2 fois dans la liste des téléchargements d'un utilisateur, donc un trigger bloquera l'action et générera une alerte.

# Requêtes possibles

## Table : Utilisateur/trice

1. Lister tous les utilisateurs qui ont l'âge requis pour visionner un contenu déconseillé au moins de 16 ans (avec date de naissance d'un utilisateur et limite d'âge d'un contenu).
2. Lister les utilisateurs inactifs n'ayant regardé aucun film ni épisode de série au cours des 90 derniers jours.

## Table : Visionne

3. Calculer l'âge moyen des utilisateurs ayant regardé au moins un épisode de série.
4. Pour l'année 2025, quelle série a été le plus visionnée (pour faire un classement annuel de chaque série).
5. Afficher les 5 films ayant obtenu le plus grand nombre de « like ».
6. Lister les utilisateurs qui ont à la fois visionné un film et visionné un épisode d'une série d'une saison.
7. Quels sont les utilisateurs qui ont visionné tous les épisodes d'une série.
8. Calculer le taux d'achèvement par utilisateur et par série (nombre d'épisodes vus / nombre total d'épisodes de la série).
9. Calculer le nombre moyen d'épisode visionné par mois selon le type d'abonnement (basique vs premium).

## Table : Episode

10. Calculer la moyenne ainsi que le maximum et le minimum de durée des épisodes de chaque série.
11. Afficher les séries dont la durée moyenne des épisodes est supérieure à 45 minutes.

## Table : Télécharge

12. Faire un classement des séries les plus téléchargées.
13. Lister les films ayant été téléchargés plus de 100 fois.

**Table : Acteur/trice**

14. Lister tous les personnages d'un acteur avec sa série.
15. Lister les acteurs ayant joué dans au moins trois séries.

**Table : Abonnement**

16. Quel abonnement est le plus vendu pour les jeunes (entre 18 et 25 ans).
17. Afficher le revenu total mensuel par type d'abonnement (nombre d'abonnements actifs × prix mensuel).

**Table : Réalisateur/trice**

18. Quels réalisateurs ont produit au minimum deux séries.

**Table : Série**

19. Lister toutes les séries, même celles qui n'ont jamais reçu de « like », ainsi que le nombre total de likes associés.

# Droits et Vues

## 1. Gestion des droits d'accès

**Administrateur**
• Accès complet à la base de données.

**Responsable du contenu**
• Peut ajouter, supprimer ou modifier les séries, films, saisons, épisodes, acteurs et réalisateurs.
• Accès en lecture seule aux statistiques globales (nombre de vues, popularité des séries, etc.).
• Aucun accès aux données personnelles des utilisateurs.
• Gère le choix de limite d'âge et la publication ou non d'un contenu.

**Service abonnements**
• Accès uniquement aux informations liées aux abonnements (type, prix, durée, ect.)
• Peut lier un abonnement à un utilisateur.
• Accès limité aux données personnelles nécessaires à la facturation (nom, prénom, adresse, ect).
• Données protégées conformément au RGPD (Règlement Général Protection des Données).

**Utilisateur basique**
• Peut visionner des séries et des films dans la limite de 50 contenus par mois.
• Ne peut pas télécharger de contenu.
• Peut liker un contenu uniquement après l'avoir visionné.
• Accès restreint à ses propres informations (profil, historique, abonnement).

**Utilisateur premium**
• Dispose de tous les droits d'un utilisateur basique.
• Peut télécharger des films et des épisodes dans une liste limité (20 contenus maximum).
• Peut consulter son historique complet et ses statistiques personnelles (nombre de visionnages, téléchargements, likes).

**Analyste / Auditeur**
• Accès en lecture seule à des données agrégées et anonymisées.
• Peut produire des rapports statistiques (taux d'abonnement, popularité des séries, activité globale).
• Aucun accès aux données personnelles identifiantes (e-mails, adresses, etc.).
• Ne peut modifier aucune donnée.

## 2. Vues associées

**Vue administrateur**
• Accès à l'ensemble des tables sans restriction.

**Vue responsable du contenu**
• Affiche les informations principales des films (titre, durée, année de sortie, disponibilité).
• Vue séries : liste des séries avec toutes les informations détaillées.
• Vue contenu : informations détaillées sur les contenus (titre, durée, date de sortie, limite d'âge).
• Vue épisodes : liste des séries avec le nombre de saisons et d'épisodes.
• Vue films : liste des films avec toutes les informations détaillées.

**Vue service abonnements**
• Vue utilisateur simplifiée : nom, prénom, type d'abonnement, date d'inscription.
• Vue client–abonnement : type d'abonnement, date de début, date de fin, statut.
• Aucune donnée sensible n'est visible (mot de passe, historique, etc.).

**Vue utilisateur**
• Vue catalogue public : séries et films accessibles à tous les utilisateurs.
• Vue historique personnel : contenus visionnés, téléchargés, likes.
• Vue compte personnel : profil, abonnement actif, et informations personnelles.

**Vue analyste / auditeur**
• Vue statistiques anonymisées : taux d'abonnement, popularité des séries, durée moyenne de visionnage, revenus mensuels estimés.
• Accès en lecture seule uniquement.

**Vue Média**

• Vue sur les informations des épisodes d'une saison d'une série

• Vue sur les informations des films

# Traduction du schéma relationnel

## 1. Tables et Attributs

Utilisateur/trice(**id_utilisateur**, nom, prenom, pays, email, mot_de_passe, date_naissance, date_inscription)

Abonnement(**id_abonnement**, type_abonnement, prix, duree)

Prends(id_utilisateur, id_abonnement, debut_abonnement)

Film(**id_film**, annee_apparition, pays_production, titre, duree, limite_age)

Serie(**id_serie**, titre, annee_apparition, pays_production, limite_age)

Episode(**id_episode**, numero_episode, numero_saison, titre, duree, id_serie)

Visionne(**id_visionnage**, date_visionnage, duree_visionnage, like, id_utilisateur, id_film, id_episode)

Telecharge(**id_telechargement**, date_telechargement, id_utilisateur, id_film, id_episode)

Acteur/trice(**id_acteur**, nom, prenom, date_naissance, nationalite, taille, poids)

Personnage(**id_personnage**, prenom, nom, type_role, description)

Incarne(id_acteur, id_personnage, version)

Realisateur/trice(**id_realisateur**, nom, prenom, date_naissance, nationalite)

Realise_Film(id_realisateur, id_film)

Realise_Serie(id_realisateur, id_serie)

Appartient(id_personnage, id_serie)

Inclus(id_personnage, id_film)

## 2. Clés étrangères

(Utilisateur/trice).id_utilisateur → Prends.id_utilisateur

(Utilisateur/trice).id_utilisateur → Visionne.id_utilisateur

(Utilisateur/trice).id_utilisateur → Telecharge.id_utilisateur

Abonnement.id_abonnement → Prends.id_abonnement

Episode.id_episode → Visionne.id_episode

Episode.id_episode → Telecharge.id_episode

Episode.id_episode → Telech.id_episode

Film.id_film → Visionne.id_film

Film.id_film → Télécharge.id_film

Film.id_film → Inclus.id_film

Film.id_film → (Realise_Film).id_film

Serie.id_serie → Episode.id_serie

Serie.id_serie → (Realise_Serie).id_serie

Serie.id_serie → Appartient.id_serie

(Realisateur/trice).id_realisateur → (Realise_Film).id_realisateur

(Realisateur/trice).id_realisateur → (Realise_Serie).id_realisateur

Personnage.id_personnage → Inclus.id_personnage

Personnage.id_personnage → Appartient.id_personnage

Personnage.id_personnage → Incarne.id_personnage

(Acteur/trice).id_acteur → Incarne.id_acteur

# PHASE 2

## Repartition du travail (1 tier du travail chacun)

Amin MEZOUER

Pio DUBRAIL

Wiam MEJJATI

## A/ Créations des tables :

```
CREATE TABLE Utilisateur (
    id_utilisateur      NUMBER CONSTRAINT pk_utilisateur PRIMARY KEY,
    nom                 VARCHAR2(50)  NOT NULL,
    prenom              VARCHAR2(50)  NOT NULL,
    pays                VARCHAR2(50)  NOT NULL,
    email               VARCHAR2(100) NOT NULL CONSTRAINT unique_email UNIQUE,
    mot_de_passe        VARCHAR2(100) NOT NULL,
    date_naissance      DATE          NOT NULL,
    date_inscription    DATE          NOT NULL)
);

CREATE TABLE Abonnement (
    id_abonnement       NUMBER CONSTRAINT pk_abonnement PRIMARY KEY,
    type_abonnement     VARCHAR2(50) NOT NULL,
    prix                NUMBER(5,2)  NOT NULL,
    duree               NUMBER       NOT NULL
);
```

```
CREATE TABLE Film (
    id_film             NUMBER CONSTRAINT pk_film PRIMARY KEY,
    titre               VARCHAR2(100) NOT NULL,
    annee_apparition    NUMBER(4)     NOT NULL,
    pays_production     VARCHAR2(50)  NOT NULL,
    duree               NUMBER        NOT NULL,
    limite_age          NUMBER        NOT NULL
);

CREATE TABLE Serie (
    id_serie            NUMBER CONSTRAINT pk_serie PRIMARY KEY,
    titre               VARCHAR2(100) NOT NULL,
    annee_apparition    NUMBER(4)     NOT NULL,
    pays_production     VARCHAR2(50)  NOT NULL,
    limite_age          NUMBER        NOT NULL
);

CREATE TABLE Personnage (
    id_personnage       NUMBER CONSTRAINT pk_personnage PRIMARY KEY,
    prenom              VARCHAR2(50) NOT NULL,
    nom                 VARCHAR2(50) NOT NULL,
    type_role           VARCHAR2(50) NOT NULL,
    description         VARCHAR2(255)
);

CREATE TABLE Realisateur (
    id_realisateur      NUMBER CONSTRAINT pk_realisateur PRIMARY KEY,
    nom                 VARCHAR2(50) NOT NULL,
    prenom              VARCHAR2(50) NOT NULL,
    date_naissance      DATE         NOT NULL,
    nationalite         VARCHAR2(50) NOT NULL
);

CREATE TABLE Prends (
    id_utilisateur      NUMBER NOT NULL,
    id_abonnement       NUMBER NOT NULL,
    debut_abonnement    DATE   NOT NULL,
    CONSTRAINT pk_prends PRIMARY KEY (id_utilisateur, id_abonnement),
    CONSTRAINT fk_prends_utilisateur FOREIGN KEY (id_utilisateur) REFERENCES
Utilisateur(id_utilisateur) ON DELETE CASCADE,
    CONSTRAINT fk_prends_abonnement FOREIGN KEY (id_abonnement) REFERENCES
Abonnement(id_abonnement) ON DELETE CASCADE
);
```

```
CREATE TABLE Incarne (
    id_acteur           NUMBER NOT NULL,
    id_personnage       NUMBER NOT NULL,
    version             VARCHAR2(50),
    CONSTRAINT pk_incarne PRIMARY KEY (id_acteur, id_personnage),
    CONSTRAINT fk_incarne_acteur FOREIGN KEY (id_acteur)
      REFERENCES Acteur(id_acteur) ON DELETE CASCADE,
    CONSTRAINT fk_incarne_personnage FOREIGN KEY (id_personnage)
      REFERENCES Personnage(id_personnage) ON DELETE CASCADE
);

CREATE TABLE Realise_Film (
    id_realisateur      NUMBER NOT NULL,
    id_film             NUMBER NOT NULL,
    CONSTRAINT pk_realisefilm PRIMARY KEY (id_realisateur, id_film),
    CONSTRAINT fk_realisefilm_realisateur FOREIGN KEY (id_realisateur)
        REFERENCES Realisateur(id_realisateur) ON DELETE CASCADE,
    CONSTRAINT fk_realisefilm_film FOREIGN KEY (id_film)
        REFERENCES Film(id_film) ON DELETE CASCADE
);

CREATE TABLE Realise_Serie (
    id_realisateur      NUMBER NOT NULL,
    id_serie            NUMBER NOT NULL,
    CONSTRAINT pk_realiseserie PRIMARY KEY (id_realisateur, id_serie),
    CONSTRAINT fk_realiseserie_realisateur FOREIGN KEY (id_realisateur)
        REFERENCES Realisateur(id_realisateur) ON DELETE CASCADE,
    CONSTRAINT fk_realiseserie_serie FOREIGN KEY (id_serie)
        REFERENCES Serie(id_serie) ON DELETE CASCADE
);

CREATE TABLE Appartient (
    id_personnage       NUMBER NOT NULL,
    id_serie            NUMBER NOT NULL,
    CONSTRAINT pk_appartient PRIMARY KEY (id_personnage, id_serie),
    CONSTRAINT fk_appartient_personnage FOREIGN KEY (id_personnage)
        REFERENCES Personnage(id_personnage) ON DELETE CASCADE,
    CONSTRAINT fk_appartient_serie FOREIGN KEY (id_serie)
        REFERENCES Serie(id_serie) ON DELETE CASCADE
);

CREATE TABLE Inclus (
    id_personnage       NUMBER NOT NULL,
    id_film             NUMBER NOT NULL,
    CONSTRAINT pk_inclus PRIMARY KEY (id_personnage, id_film),
    CONSTRAINT fk_inclus_personnage FOREIGN KEY (id_personnage)
        REFERENCES Personnage(id_personnage) ON DELETE CASCADE,
    CONSTRAINT fk_inclus_film FOREIGN KEY (id_film)
        REFERENCES Film(id_film) ON DELETE CASCADE
);
```

```
CREATE TABLE Visionne (
    id_visionnage       NUMBER CONSTRAINT pk_visionne PRIMARY KEY,
    date_visionnage     DATE   NOT NULL,
    duree_visionnage    NUMBER NOT NULL,
    likes               NUMBER(1) NOT NULL,
    id_utilisateur      NUMBER NOT NULL,
    id_film             NUMBER,
    id_episode          NUMBER,
    CONSTRAINT fk_visionne_utilisateur FOREIGN KEY (id_utilisateur)
        REFERENCES Utilisateur(id_utilisateur) ON DELETE CASCADE,
    CONSTRAINT fk_visionne_film FOREIGN KEY (id_film)
        REFERENCES Film(id_film) ON DELETE CASCADE,
    CONSTRAINT fk_visionne_episode FOREIGN KEY (id_episode)
        REFERENCES Episode(id_episode) ON DELETE CASCADE
);

CREATE TABLE Telecharge (
    id_telechargement   NUMBER CONSTRAINT pk_telecharge PRIMARY KEY,
    date_telechargement DATE NOT NULL,
    id_utilisateur      NUMBER NOT NULL,
    id_film             NUMBER,
    id_episode          NUMBER,
    CONSTRAINT fk_telecharge_utilisateur FOREIGN KEY (id_utilisateur)
        REFERENCES Utilisateur(id_utilisateur) ON DELETE CASCADE,
    CONSTRAINT fk_telecharge_film FOREIGN KEY (id_film)
        REFERENCES Film(id_film) ON DELETE CASCADE,
    CONSTRAINT fk_telecharge_episode FOREIGN KEY (id_episode)
        REFERENCES Episode(id_episode) ON DELETE CASCADE
);
```

## Insertion des sequences :

Utile pour l'insertion des données, pour éviter les conflits de clé primaires. Nous allons créer des sequences qui vont incrémenter de 1 de manière automatique les clé primaires (qui sont tous des NUMBER) de toutes nos tables lors d'une insertion de ligne sur la table correspondante.

```
CREATE SEQUENCE seq_utilisateur  START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_abonnement   START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_film         START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_serie        START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_episode      START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_visionne     START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_telecharge   START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_realisateur  START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_acteur       START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_personnage   START WITH 1 INCREMENT BY 1;
```

### Exemple insertion de valeur avec sequence (avec NEXTVAL) :

```
INSERT INTO Film VALUES (seq_film.NEXTVAL, 'Film_test', 2020, 'France', 120, 12);
```

## D/ Contraintes d'attribut & d'intégrité complexe

### 1) Création des checks (contraintes d'attribut) :

```
ALTER TABLE Utilisateur
ADD CONSTRAINT ck_utilisateur_date_naissance
CHECK (date_naissance < date_inscription);

ALTER TABLE Abonnement
ADD CONSTRAINT ck_abonnement_prix
CHECK (prix > 0);

ALTER TABLE Film
ADD CONSTRAINT ck_film_limite_age
CHECK (limite_age IN (0, 7, 12, 16, 18));

ALTER TABLE Serie
ADD CONSTRAINT ck_serie_limite_age
CHECK (limite_age IN (0, 7, 12, 16, 18));
```

```
ALTER TABLE Visionne
ADD CONSTRAINT ck_visionne_film_ou_episode
CHECK ((id_film IS NOT NULL AND id_episode IS NULL)
  OR (id_film IS NULL AND id_episode IS NOT NULL));

ALTER TABLE Telecharge
ADD CONSTRAINT ck_telecharge_film_ou_episode
CHECK ((id_film IS NOT NULL AND id_episode IS NULL)
  OR (id_film IS NULL AND id_episode IS NOT NULL));

ALTER TABLE Visionne
ADD CONSTRAINT ck_visionne_likes
CHECK (likes IN (0,1));

ALTER TABLE Episode
ADD CONSTRAINT ck_episode_num_saison
CHECK (numero_saison >= 1);

ALTER TABLE Episode
ADD CONSTRAINT ck_episode_num_episode
CHECK (numero_episode >= 1);
```

## 2) Création des triggers (contraintes complexes) :

```
CREATE OR REPLACE TRIGGER visionne_abonnement_actif

BEFORE INSERT ON Visionne

FOR EACH ROW

DECLARE

    nombre_visionnage NUMBER;

BEGIN

    SELECT COUNT(*)

    INTO nombre_visionnage

    FROM Prends p

        JOIN Abonnement a ON a.id_abonnement = p.id_abonnement

    WHERE p.id_utilisateur = :NEW.id_utilisateur

    AND :NEW.date_visionnage >= p.debut_abonnement

    -- ADD_MONTHS permet d'ajouter un nombre de mois (a specifié) à un attribut de
type DATE

    AND :NEW.date_visionnage < ADD_MONTHS(p.debut_abonnement, a.duree);


    IF nombre_visionnage = 0 THEN

RAISE_APPLICATION_ERROR(-20000,'Aucun abonnement actif à la date de visionnage.');

    END IF;

END;

/
```

```
CREATE OR REPLACE TRIGGER visionne_limite_basique
BEFORE INSERT ON Visionne
FOR EACH ROW
DECLARE
    nb_basique NUMBER;
    nb_mois NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO nb_basique
    FROM Prends p
           JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
    WHERE p.id_utilisateur = :NEW.id_utilisateur
    AND a.type_abonnement = 'Basique'
    AND :NEW.date_visionnage >= p.debut_abonnement
    AND :NEW.date_visionnage < ADD_MONTHS(p.debut_abonnement, a.duree);
    IF nb_basique > 0 THEN
        SELECT COUNT(*) INTO nb_mois
        FROM Visionne v
        WHERE v.id_utilisateur = :NEW.id_utilisateur
        AND EXTRACT(YEAR  FROM v.date_visionnage) = EXTRACT(YEAR  FROM
:NEW.date_visionnage)
        AND EXTRACT(MONTH FROM v.date_visionnage) = EXTRACT(MONTH FROM
:NEW.date_visionnage);
        IF nb_mois >= 50 THEN
            RAISE_APPLICATION_ERROR(-20001,
                'Limite de 50 visionnages par mois atteinte (abonnement
basique).');
        END IF;
    END IF;
END;
/
```

```
CREATE OR REPLACE TRIGGER visionne_limite_age
BEFORE INSERT ON Visionne
FOR EACH ROW
DECLARE
    t_naiss DATE;
    t_age NUMBER;
    t_limite NUMBER;
BEGIN
    SELECT date_naissance INTO t_naiss
    FROM Utilisateur
    WHERE id_utilisateur = :NEW.id_utilisateur;

    t_age := FLOOR(MONTHS_BETWEEN(:NEW.date_visionnage, t_naiss) / 12);

    IF :NEW.id_film IS NOT NULL THEN
        SELECT limite_age INTO t_limite FROM Film WHERE id_film = :NEW.id_film;
    ELSE
        SELECT s.limite_age
        INTO t_limite
        FROM Episode e JOIN Serie s ON s.id_serie = e.id_serie
        WHERE e.id_episode = :NEW.id_episode;
    END IF;

    IF t_age < t_limite THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Accès refusé : limite d''âge non respectée.');
    END IF;
END;
/
```

```
CREATE OR REPLACE TRIGGER telecharge_abonnement_actif
BEFORE INSERT ON Telecharge
FOR EACH ROW
DECLARE
    t_nb NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO t_nb
    FROM Prends p
           JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
    WHERE p.id_utilisateur = :NEW.id_utilisateur
    AND :NEW.date_telechargement >= p.debut_abonnement
    AND :NEW.date_telechargement < ADD_MONTHS(p.debut_abonnement, a.duree);

    IF t_nb = 0 THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Aucun abonnement actif à la date de téléchargement.');
    END IF;
END;
/
```

```
CREATE OR REPLACE TRIGGER telechargement_premium
BEFORE INSERT ON Telecharge
FOR EACH ROW
DECLARE
    v_nb_premium NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO   v_nb_premium
    FROM   Prends p
           JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
    WHERE  p.id_utilisateur = :NEW.id_utilisateur
    AND    a.type_abonnement = 'Premium'
    AND    :NEW.date_telechargement >= p.debut_abonnement
    AND    :NEW.date_telechargement < ADD_MONTHS(p.debut_abonnement, a.duree);

    IF v_nb_premium = 0 THEN
        RAISE_APPLICATION_ERROR(-20004,
            'Téléchargement interdit : abonnement Premium requis.');
    END IF;
END;
/
```

```
CREATE OR REPLACE TRIGGER telechargement_max20
BEFORE INSERT ON Telecharge
FOR EACH ROW
DECLARE
    v_nb     NUMBER;
    v_min_dt DATE;
    v_min_id NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO   v_nb
    FROM   Telecharge
    WHERE  id_utilisateur = :NEW.id_utilisateur;
    IF v_nb >= 20 THEN
        SELECT MIN(date_telechargement)
        INTO   v_min_dt
        FROM   Telecharge
        WHERE  id_utilisateur = :NEW.id_utilisateur;
        SELECT MIN(id_telechargement)
        INTO   v_min_id
        FROM   Telecharge
        WHERE  id_utilisateur = :NEW.id_utilisateur
        AND    date_telechargement = v_min_dt;

        DELETE FROM Telecharge
        WHERE id_telechargement = v_min_id;
    END IF;
END;
/
```

## B/ Insertion des données :

### Insertion manuelle :

```
INSERT INTO Abonnement VALUES (seq_abonnement.NEXTVAL, 'Basique', 5.99, 1);

INSERT INTO Abonnement VALUES (seq_abonnement.NEXTVAL, 'Premium', 12.99, 1);


INSERT INTO Utilisateur VALUES (seq_utilisateur.NEXTVAL, 'MEZOUER', 'Amîn',
'France', 'amin.mezouer@ens.uvsq.fr', '*******', TO_DATE('2004-05-12','YYYY-MM-
DD'), SYSDATE);


INSERT INTO Film VALUES (seq_film.NEXTVAL,'Arsène Lupin',2020,'France',120,12);


INSERT INTO Serie VALUES (seq_serie.NEXTVAL,'Validé 2',2022,'France',16);

COMMIT; --Pour enregistrement des changements—
```

### Insertion massive de donnée (SQL*Loader) :

Nous avons fait le choix de créer un fichier python qui génère des fichiers CSV contenant des données aléatoires mais cohérent avec la réalité pour les tables qui peuvent entrer en conflit avec les triggers que nous avons créé. Pour les autres tables nous avons fait des insertions manuelles mais robustes pour entrer en cohérence avec les autres tables.

```
python generation_donnee.py
```

Nous avons fait cette commande sur notre terminal de la VM, et ça nous a générer des fichiers CSV correspondant à l’insertion de certaines tables comme : Prends.csv Utilisateur.csv Visionne.csv Telecharge.csv Film.csv Serie.csv Episode.csv.

Fichier Python qui assure ces generations de fichier CSV :

```
head -n 10 generation_donnee.py
```

Pour afficher les 10 premières lignes du fichier python car le fichier est très long (nous avons dû réadapter plusieurs fois notre code python car la commande python dans le terminal de la VM reconnait une version très ancienne de python : python 2.4). Le code complet du script python sera en annexe du projet (dernières pages)

```
[oracle@localhost data]$ head -n 10 generation_donnee.py
import csv
import random
from datetime import date, timedelta

random.seed(42)

N_USERS = 150
N_FILMS = 200
N_SERIES = 60
N_EPISODES = 300
[oracle@localhost data]$
```

Création des fichiers .ctl après exécution du script python :

Serie.ctl

```
OPTIONS (SKIP=1)

LOAD DATA

INFILE 'serie.csv'

APPEND

INTO TABLE Serie

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

TRAILING NULLCOLS

(id_serie INTEGER EXTERNAL, titre CHAR(100), annee_apparition INTEGER EXTERNAL,
pays_production CHAR(50), limite_age INTEGER EXTERNAL)
```

Film.ctl

```
OPTIONS (SKIP=1)
LOAD DATA
INFILE 'film.csv'
APPEND
INTO TABLE Film
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
TRAILING NULLCOLS
(
  id_film          INTEGER EXTERNAL,
  titre            CHAR(100),
  annee_apparition INTEGER EXTERNAL,
  pays_production  CHAR(50),
  duree            INTEGER EXTERNAL,
  limite_age       INTEGER EXTERNAL
)
```

Episode.ctl

```
OPTIONS (SKIP=1)
LOAD DATA
INFILE 'episode.csv'
APPEND
INTO TABLE Episode
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
TRAILING NULLCOLS
(id_episode INTEGER EXTERNAL, numero_episode INTEGER EXTERNAL, numero_saison
INTEGER EXTERNAL, id_serie INTEGER EXTERNAL, duree INTEGER EXTERNAL)
```

Utilisateur.ctl

```
OPTIONS (SKIP=1)
LOAD DATA
INFILE 'utilisateur.csv'
APPEND
INTO TABLE Utilisateur
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
(id_utilisateur, nom, prenom, pays, email, mot_de_passe, date_naissance
DATE "YYYY-MM-DD", date_inscription DATE "YYYY-MM-DD")
```

Prends.ctl

```
OPTIONS (SKIP=1)
LOAD DATA
INFILE 'prends.csv'
APPEND
INTO TABLE Prends
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
(id_utilisateur INTEGER EXTERNAL, id_abonnement INTEGER EXTERNAL,
debut_abonnement DATE "YYYY-MM-DD")
```

Visionne.ctl

```
OPTIONS (SKIP=1)

LOAD DATA

INFILE 'visionne.csv'

APPEND

INTO TABLE Visionne

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

TRAILING NULLCOLS

(id_visionnage INTEGER EXTERNAL, date_visionnage DATE "YYYY-MM-DD",
duree_visionnage INTEGER EXTERNAL, likes INTEGER EXTERNAL, id_utilisateur INTEGER
EXTERNAL, id_film INTEGER EXTERNAL NULLIF id_film=BLANKS, id_episode INTEGER
EXTERNAL NULLIF id_episode=BLANKS)
```

Telecharge.ctl

```
OPTIONS (SKIP=1)

LOAD DATA

INFILE 'telecharge.csv'

APPEND

INTO TABLE Telecharge

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

TRAILING NULLCOLS

(id_telechargement INTEGER EXTERNAL, date_telechargement DATE "YYYY-MM-DD",
id_utilisateur INTEGER EXTERNAL, id_film INTEGER EXTERNAL NULLIF id_film=BLANKS,
id_episode INTEGER EXTERNAL NULLIF id_episode=BLANKS)
```

Puis je charge ce qu'il y a dans mes fichiers CTL avec les commandes suivantes : (l'ordre a une importance)

```
sqlldr userid=SYSTEM/oracle control=abonnement.ctl log=abonnement.log

sqlldr userid=SYSTEM/oracle control=utilisateur.ctl log=utilisateur.log

sqlldr userid=SYSTEM/oracle control=film.ctl log=film.log

sqlldr userid=SYSTEM/oracle control=serie.ctl log=serie.log

sqlldr userid=SYSTEM/oracle control=episode.ctl log=episode.log

sqlldr userid=SYSTEM/oracle control=prends.ctl log=prends.log

sqlldr userid=SYSTEM/oracle control=visionne.ctl log=visionne.log

sqlldr userid=SYSTEM/oracle control=telecharge.ctl log=telecharge.log
```

<u>Affichage logique :</u>

```
[oracle@localhost data]$ sqlldr userid=SYSTEM/oracle control=visionne.ctl lo
g=visionne.log

SQL*Loader: Release 11.2.0.2.0 - Production on Sun Dec 14 21:40:48 2025

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved
.

Commit point reached - logical record count 64
Commit point reached - logical record count 128
Commit point reached - logical record count 192
Commit point reached - logical record count 256
Commit point reached - logical record count 320
Commit point reached - logical record count 384
Commit point reached - logical record count 448
Commit point reached - logical record count 512
Commit point reached - logical record count 576
Commit point reached - logical record count 640
Commit point reached - logical record count 704
Commit point reached - logical record count 768
Commit point reached - logical record count 832
Commit point reached - logical record count 896
Commit point reached - logical record count 960
Commit point reached - logical record count 1000
[oracle@localhost data]$ sqlldr userid=SYSTEM/oracle control=telecharge.ctl
log=telecharge.log

SQL*Loader: Release 11.2.0.2.0 - Production on Sun Dec 14 21:41:13 2025

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved
.

Commit point reached - logical record count 64
Commit point reached - logical record count 128
```

Afin de vérifier le bon chargement des données dans la base, nous effectuons des requêtes de comptage sur chaque table.

```
SELECT 'UTILISATEUR' AS table_nom, COUNT(*) AS nb FROM Utilisateur
UNION ALL
SELECT 'ABONNEMENT', COUNT(*) FROM Abonnement
UNION ALL
SELECT 'FILM', COUNT(*) FROM Film
UNION ALL
SELECT 'SERIE', COUNT(*) FROM Serie
UNION ALL
SELECT 'EPISODE', COUNT(*) FROM Episode
UNION ALL
SELECT 'PRENDS', COUNT(*) FROM Prends
UNION ALL
SELECT 'VISIONNE', COUNT(*) FROM Visionne
UNION ALL
SELECT 'TELECHARGE', COUNT(*) FROM Telecharge;
```

<u>Affichage logique :</u>

```
TABLE_NOM   NB
----------- --
UTILISATEUR 150
ABONNEMENT   2
FILM        200
SERIE       60
EPISODE     300
PRENDS      150
VISIONNE    965
TELECHARGE  150
 8 lignes sélectionnées
```

Certains Trigger bloque l'insertion de certaines lignes c'est pour cela que nous n'avons pas forcément toutes les lignes inséré pour Visionne et Télécharge. Dans notre script python nous avons volontairement généré plus de prise d'abonnement 'Premium' pour justement avoir moins de contrainte de visionnage ou téléchargement.

## C/ Requêtes :

**Table : Utilisateur/trice**

1. Lister tous les utilisateurs qui ont l'âge requis pour visionner un contenu déconseillé au moins de 16 ans (avec date de naissance d'un utilisateur et limite d'âge d'un contenu).

```
SELECT DISTINCT u.id_utilisateur, u.nom, u.prenom
FROM Utilisateur u
JOIN Visionne v ON v.id_utilisateur = u.id_utilisateur
LEFT JOIN Film f ON f.id_film = v.id_film
LEFT JOIN Episode e ON e.id_episode = v.id_episode
LEFT JOIN Serie s ON s.id_serie = e.id_serie
WHERE NVL(f.limite_age, s.limite_age) >= 16
  AND MONTHS_BETWEEN(v.date_visionnage, u.date_naissance)/12 >= 16;
```

2. Lister les utilisateurs inactifs n'ayant regardé aucun film ni épisode de série au cours des 90 derniers jours.

```
SELECT u.id_utilisateur, u.nom, u.prenom
FROM Utilisateur u
LEFT JOIN Visionne v
  ON v.id_utilisateur = u.id_utilisateur
 AND v.date_visionnage >= SYSDATE - 90
WHERE v.id_visionnage IS NULL;
```

**Table : Visionne**

3. Calculer l'âge moyen des utilisateurs ayant regardé au moins un épisode de série.

```
SELECT AVG(MONTHS_BETWEEN(SYSDATE, u.date_naissance)/12) AS age_moyen
FROM Utilisateur u
JOIN Visionne v ON v.id_utilisateur = u.id_utilisateur
WHERE v.id_episode IS NOT NULL;
```

4. Pour l'année 2025, quelle série a été le plus visionnée (pour faire un classement annuel de chaque série).

```
SELECT titre, nb_visionnages
FROM (
  SELECT s.titre, COUNT(*) AS nb_visionnages
  FROM Visionne v
  JOIN Episode e ON e.id_episode = v.id_episode
  JOIN Serie s   ON s.id_serie = e.id_serie
  WHERE EXTRACT(YEAR FROM v.date_visionnage) = 2025
  GROUP BY s.titre
  ORDER BY nb_visionnages DESC
)
WHERE ROWNUM = 1;
```

5. Afficher les 5 films ayant obtenu le plus grand nombre de « like ».

```
SELECT * FROM ( SELECT f.titre, SUM(v.likes) AS total_likes
                FROM Visionne v JOIN Film f ON f.id_film = v.id_film
                GROUP BY f.titre
                ORDER BY total_likes DESC)
WHERE ROWNUM <= 5;
```

6. Lister les utilisateurs qui ont à la fois visionné un film et visionné un épisode d'une série d'une saison.

```
SELECT u.id_utilisateur, u.nom, u.prenom

FROM Utilisateur u

WHERE EXISTS (SELECT 1 FROM Visionne v WHERE v.id_utilisateur=u.id_utilisateur AND
v.id_film IS NOT NULL)

  AND EXISTS (SELECT 1 FROM Visionne v WHERE v.id_utilisateur=u.id_utilisateur AND
v.id_episode IS NOT NULL);
```

7. Quels sont les utilisateurs qui ont visionné tous les épisodes d'une série.

```
SELECT v.id_utilisateur, e.id_serie

FROM Visionne v

JOIN Episode e ON e.id_episode = v.id_episode

GROUP BY v.id_utilisateur, e.id_serie

HAVING COUNT(DISTINCT v.id_episode) =

       (SELECT COUNT(*) FROM Episode ep WHERE ep.id_serie = e.id_serie);
```

8. Calculer le taux d'achèvement par utilisateur et par série (nombre d'épisodes vus / nombre total d'épisodes de la série).

```
SELECT v.id_utilisateur,

       e.id_serie,

       COUNT(DISTINCT v.id_episode)

       / (SELECT COUNT(*) FROM Episode ep WHERE ep.id_serie = e.id_serie) AS taux

FROM Visionne v

JOIN Episode e ON e.id_episode = v.id_episode

GROUP BY v.id_utilisateur, e.id_serie;
```

9. Calculer le nombre moyen d'épisode visionné par mois selon le type d'abonnement (basique vs premium).

```
SELECT type_abonnement, AVG(nb_episodes) AS moyenne_par_mois
FROM ( SELECT a.type_abonnement,
         EXTRACT(YEAR FROM v.date_visionnage) AS annee,
         EXTRACT(MONTH FROM v.date_visionnage) AS mois,
         COUNT(*) AS nb_episodes
  FROM Visionne v
  JOIN Prends p ON p.id_utilisateur = v.id_utilisateur
  JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
  WHERE v.id_episode IS NOT NULL
  GROUP BY a.type_abonnement,
           EXTRACT(YEAR FROM v.date_visionnage),
           EXTRACT(MONTH FROM v.date_visionnage)
)
GROUP BY type_abonnement;
```

**Table : Episode**

10. Calculer la moyenne ainsi que le maximum et le minimum de durée des épisodes de chaque série.

```
SELECT id_serie, AVG(duree) AS duree_moyenne,
       MAX(duree) AS duree_max,
       MIN(duree) AS duree_min
FROM Episode
GROUP BY id_serie;
```

11. Afficher les séries dont la durée moyenne des épisodes est supérieure à 45 minutes.

```
SELECT id_serie
FROM Episode
GROUP BY id_serie
HAVING AVG(duree) > 45;
```

**Table : Télécharge**

12. Faire un classement des séries les plus téléchargées.

```
SELECT e.id_serie, COUNT(*) AS nb_telechargements
FROM Telecharge t
JOIN Episode e ON e.id_episode = t.id_episode
GROUP BY e.id_serie
ORDER BY nb_telechargements DESC;
```

13. Lister les films ayant été téléchargés plus de 100 fois.

```
SELECT id_film, COUNT(*) AS nb_telechargements
FROM Telecharge
WHERE id_film IS NOT NULL
GROUP BY id_film
HAVING COUNT(*) > 100;
```

**Table : Acteur/trice**

14. Lister tous les personnages d'un acteur avec sa série.

```
SELECT
       a.id_acteur, a.nom, a.prenom,
       p.id_personnage, p.prenom AS prenom_personnage, p.nom AS nom_personnage,
       s.id_serie, s.titre AS titre_serie
FROM Acteur a
JOIN Incarne i
       ON i.id_acteur = a.id_acteur
JOIN Personnage p
       ON p.id_personnage = i.id_personnage
LEFT JOIN Appartient ap
       ON ap.id_personnage = p.id_personnage
LEFT JOIN Serie s
       ON s.id_serie = ap.id_serie
WHERE a.id_acteur = 1
ORDER BY s.titre, p.nom, p.prenom;
```

15. Lister les acteurs ayant joué dans au moins trois séries.

```
SELECT a.id_acteur, a.nom, a.prenom, COUNT(DISTINCT ap.id_serie) AS nb_series
FROM Acteur a
JOIN Incarne i ON i.id_acteur = a.id_acteur
JOIN Appartient ap ON ap.id_personnage = i.id_personnage
GROUP BY a.id_acteur, a.nom, a.prenom
HAVING COUNT(DISTINCT ap.id_serie) >= 3;
```

## Table : Abonnement

16. Quel abonnement est le plus vendu pour les jeunes (entre 18 et 25 ans).

```
SELECT type_abonnement, nb
FROM (SELECT a.type_abonnement, COUNT(*) AS nb
  FROM Prends p
  JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
  JOIN Utilisateur u ON u.id_utilisateur = p.id_utilisateur
  WHERE MONTHS_BETWEEN(SYSDATE, u.date_naissance)/12 BETWEEN 18 AND 25
  GROUP BY a.type_abonnement
  ORDER BY nb DESC
)
WHERE ROWNUM = 1;
```

17. Afficher le revenu total mensuel par type d'abonnement (nombre d'abonnements actifs × prix mensuel).

```
SELECT a.type_abonnement, COUNT(*) * a.prix AS revenu
FROM Prends p
JOIN Abonnement a ON a.id_abonnement = p.id_abonnement
WHERE SYSDATE >= p.debut_abonnement
  AND SYSDATE < ADD_MONTHS(p.debut_abonnement, a.duree)
GROUP BY a.type_abonnement, a.prix;
```

## Table : Réalisateur/trice

18. Quels réalisateurs ont produit au minimum deux séries.

```
SELECT r.id_realisateur, r.nom, r.prenom, COUNT(*) AS nb_series
FROM Realisateur r
JOIN Realise_Serie rs ON rs.id_realisateur = r.id_realisateur
GROUP BY r.id_realisateur, r.nom, r.prenom
HAVING COUNT(*) >= 2;
```

## Table : Série

19. Lister toutes les séries, même celles qui n'ont jamais reçu de « like », ainsi que le nombre total de likes associés.

```
SELECT s.id_serie, s.titre, SUM(v.likes) AS total_likes
FROM Serie s
LEFT JOIN Episode e ON e.id_serie = s.id_serie
LEFT JOIN Visionne v ON v.id_episode = e.id_episode
GROUP BY s.id_serie, s.titre;
```

## C/ Droits et Vues :

### 1er Etape : Création des rôles (pour leurs mettrent des droits) :

```
CREATE ROLE role_resp_contenu;

CREATE ROLE role_service_abos;

CREATE ROLE role_utilisateur_basique;

CREATE ROLE role_utilisateur_premium;

CREATE ROLE role_analyste;
```

### 2eme Etape : Création des Vues :

**Catalogue Public (Vue sur Film et Série)**

Cette vue présente le catalogue public des contenus disponibles sur la plateforme.
Elle liste l'ensemble des films et des séries avec leur titre, leur année de sortie et leur limite d'âge, sans information personnelle sur les utilisateurs.

```
CREATE OR REPLACE VIEW v_catalogue_public AS

SELECT f.id_film AS id_contenu, f.titre, f.limite_age, f.annee_apparition, 'FILM'
AS type_contenu

FROM Film f

UNION ALL

SELECT s.id_serie AS id_contenu, s.titre, s.limite_age, s.annee_apparition,
'SERIE' AS type_contenu

FROM Serie s;
```

**Vue Historique (Visionne + Télécharge + Titre du contenu)**

Cette vue fournit l'historique des activités d'un utilisateur sur la plateforme.
Elle regroupe les visionnages et les téléchargements effectués par chaque utilisateur, en précisant le contenu concerné (film ou épisode de série), la date et le nombre de likes associés.

```
CREATE OR REPLACE VIEW v_historique AS


-- Historique des visionnages --

SELECT v.id_utilisateur, v.date_visionnage AS date_action, 'VISIONNAGE' AS
type_action, v.likes, f.titre AS titre_film, s.titre AS titre_serie,
e.numero_saison, e.numero_episode

FROM Visionne v

LEFT JOIN Film f ON f.id_film = v.id_film

LEFT JOIN Episode e ON e.id_episode = v.id_episode

LEFT JOIN Serie s ON s.id_serie = e.id_serie


UNION ALL


-- Historique des téléchargements

SELECT t.id_utilisateur, t.date_telechargement AS date_action, 'TELECHARGEMENT' AS
type_action, NULL AS likes, f.titre AS titre_film, s.titre AS titre_serie,
e.numero_saison, e.numero_episode

FROM Telecharge t

LEFT JOIN Film f     ON f.id_film = t.id_film

LEFT JOIN Episode e  ON e.id_episode = t.id_episode

LEFT JOIN Serie s    ON s.id_serie = e.id_serie;
```

**Vue Compte (Utilisateur + Abonnement)**

Cette vue permet à un utilisateur de consulter les informations de son compte personnel, incluant ses données d'identité ainsi que les informations liées à son abonnement actuel (type, prix, durée, date de début).

```
CREATE OR REPLACE VIEW v_compte AS

SELECT u.id_utilisateur, u.nom, u.prenom, u.email, u.pays, u.date_naissance,
u.date_inscription, a.type_abonnement, a.prix, a.duree, p.debut_abonnement

FROM Utilisateur u

LEFT JOIN Prends p ON p.id_utilisateur = u.id_utilisateur

LEFT JOIN Abonnement a ON a.id_abonnement = p.id_abonnement;
```

**Vue Responsable Contenu (Film + Série + nombre d'épisode)**

Cette vue est destinée aux responsables du contenu.
Elle permet de visualiser l'ensemble des films et séries disponibles sur la plateforme, avec leurs caractéristiques principales ainsi que, pour les séries, le nombre total d'épisodes.

```
CREATE OR REPLACE VIEW v_resp_contenu AS

SELECT 'FILM' AS type_contenu, f.id_film AS id_contenu, f.titre,
f.annee_apparition, f.pays_production, f.limite_age, NULL AS nb_episodes

FROM Film f

UNION ALL

SELECT 'SERIE', s.id_serie,s.titre,s.annee_apparition,s.pays_production,
s.limite_age, COUNT(e.id_episode) AS nb_episodes

FROM Serie s

LEFT JOIN Episode e ON e.id_serie = s.id_serie

GROUP BY s.id_serie, s.titre, s.annee_apparition, s.pays_production, s.limite_age;
```

### Vue service abonnement (Vue sur les abonnements actifs)

Cette vue est destinée au service des abonnements. Elle permet de consulter les informations des utilisateurs abonnés ainsi que les détails de leur abonnement (type, prix, durée et période de validité).

```
CREATE OR REPLACE VIEW v_service_abos AS
SELECT
  u.id_utilisateur, u.nom, u.prenom, u.email,
  a.type_abonnement, a.prix, a.duree,
  p.debut_abonnement
FROM Utilisateur u
JOIN Prends p     ON p.id_utilisateur = u.id_utilisateur
JOIN Abonnement a ON a.id_abonnement  = p.id_abonnement
WHERE SYSDATE >= p.debut_abonnement
  AND SYSDATE < ADD_MONTHS(p.debut_abonnement, a.duree);
```

### Vue Stats Analyste (Vue sur les données avec utilisation d'agregat)

Cette vue fournit des statistiques globales par type d'abonnement. Elle permet d'analyser le nombre d'abonnés, le volume de visionnages, de téléchargements et le nombre total de likes, afin d'évaluer l'activité et la rentabilité de chaque abonnement.

```
CREATE OR REPLACE VIEW v_stats AS
SELECT
  a.type_abonnement,
  COUNT(DISTINCT p.id_utilisateur) AS nb_abonnes,
  COUNT(v.id_visionnage) AS nb_visionnages,
  COUNT(t.id_telechargement) AS nb_telechargements,
  NVL(SUM(v.likes), 0) AS total_likes
FROM Abonnement a
LEFT JOIN Prends p ON p.id_abonnement = a.id_abonnement
LEFT JOIN Visionne v ON v.id_utilisateur = p.id_utilisateur
LEFT JOIN Telecharge t ON t.id_utilisateur = p.id_utilisateur
GROUP BY a.type_abonnement;
```

3eme Etape : Attribution des vues pour les rôles :

```
GRANT SELECT ON v_catalogue_public TO role_utilisateur_basique;
GRANT SELECT ON v_compte           TO role_utilisateur_basique;
GRANT SELECT ON v_catalogue_public TO role_utilisateur_premium;
GRANT SELECT ON v_compte           TO role_utilisateur_premium;
GRANT SELECT ON v_historique       TO role_utilisateur_premium;
GRANT SELECT ON v_resp_contenu     TO role_resp_contenu;
GRANT SELECT ON v_service_abos     TO role_service_abos;
GRANT SELECT ON v_stats            TO role_analyste;
```

## F/ Méta-données :

**Meta-données : liste_ora_constraint.sql**

```
SELECT
  c.table_name,
  c.constraint_name,
  c.constraint_type,
  DBMS_METADATA.GET_DDL('CONSTRAINT', c.constraint_name) AS ddl_constraint
FROM user_constraints c
ORDER BY c.table_name, c.constraint_type, c.constraint_name;
```

**Meta-données : liste_ora_triggers.sql**

```
SELECT
  t.table_name,
  t.trigger_name,
  t.status,
  DBMS_METADATA.GET_DDL('TRIGGER', t.trigger_name) AS ddl_trigger
FROM user_triggers t
ORDER BY t.table_name, t.trigger_name;
```

**Meta-données : liste_ora_views.sql**

```
SELECT
  v.view_name,
  DBMS_METADATA.GET_DDL('VIEW', v.view_name) AS ddl_view
FROM user_views v
ORDER BY v.view_name;
```

Exemple de retour (liste_ora_triggers.sql) :

| | | |
|---|---|---|
| TELECHARGE_ABONNEMENT_ACTIF | TELECHARGE | INSERT |
| TELECHARGEMENT_MAX20 | TELECHARGE | INSERT |
| TELECHARGEMENT_PREMIUM | TELECHARGE | INSERT |
| VISIONNE_ABONNEMENT_ACTIF | VISIONNE | INSERT |
| VISIONNE_LIMITE_AGE | VISIONNE | INSERT |
| VISIONNE_LIMITE_BASIQUE | VISIONNE | INSERT |

## Annexes :

```
import csv

import random

from datetime import date, timedelta

random.seed(42)

N_USERS = 150

N_FILMS = 200

N_SERIES = 60

N_EPISODES = 300

N_VISIONNAGES = 1000

N_TELECH = 300

ABO_START = date(2025, 12, 1)

ABO_END   = date(2025, 12, 31)

START_2017 = date(2017, 1, 1)

END_2017   = date(2017, 12, 31)

PCT_2017   = 0.20  # 20% des visionnages

NULL = ""

def fmt(d):

    return d.strftime("%Y-%m-%d")

def rand_date(d1, d2):

    return d1 + timedelta(days=random.randint(0, (d2 - d1).days))

def age_on(birth, d):

    a = d.year - birth.year

    if (d.month, d.day) < (birth.month, birth.day):

        a -= 1

    return a

user_birth = {}

user_plan = {}

# Repartition 70% premium / 30% basique

for u in range(1, N_USERS + 1):

    age = random.randint(13, 70)

    user_birth[u] = date(2025 - age, random.randint(1, 12), random.randint(1, 28))

    user_plan[u] = 2 if random.random() < 0.70 else 1

with open("prends.csv", "w", newline="", encoding="utf-8") as f:
```

```
    w = csv.writer(f)
    w.writerow(["id_utilisateur", "id_abonnement", "debut_abonnement"])
    for u in range(1, N_USERS + 1):
        w.writerow([u, user_plan[u], fmt(ABO_START)])
limites = [0, 7, 12, 16, 18]
pays = ["France", "USA", "UK", "Japon", "Canada"]
film_limit = {}
with open("film.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["id_film", "titre", "annee_apparition", "pays_production", "duree", "limite_age"])
    for i in range(1, N_FILMS + 1):
        la = random.choice(limites)
        film_limit[i] = la
        annee = random.choice([2016, 2017, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025])
        w.writerow([i, f"Film_{i}", annee, random.choice(pays), random.randint(70, 180), la])
serie_limit = {}
with open("serie.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["id_serie", "titre", "annee_apparition", "pays_production", "limite_age"])
    for i in range(1, N_SERIES + 1):
        la = random.choice(limites)
        serie_limit[i] = la
        annee = random.choice([2016, 2017, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025])
        w.writerow([i, f"Serie_{i}", annee, random.choice(pays), la])
pairs = set()
episodes = []  # (id_episode, id_serie)
eid = 1
f = open("episode.csv", "w", newline="", encoding="utf-8")
w = csv.writer(f)
w.writerow(["id_episode", "numero_episode", "numero_saison", "id_serie", "duree"])
while len(pairs) < N_EPISODES:
    sid = random.randint(1, N_SERIES)
    saison = random.randint(1, 5)
    ep = random.randint(1, 20)
    key = (sid, saison, ep)
    if key in pairs:
        continue
```

```
    pairs.add(key)

    duree = random.randint(20, 65)  # durée épisode réaliste

    w.writerow([eid, ep, saison, sid, duree])

    episodes.append((eid, sid))

    eid += 1

basique_count_dec2025 = {u: 0 for u in range(1, N_USERS + 1)}

star_films = [1, 2, 3, 4, 5]

star_series = [1, 2, 3, 4, 5]

with open("visionne.csv", "w", newline="", encoding="utf-8") as f:

    w = csv.writer(f)

    w.writerow(["id_visionnage", "date_visionnage", "duree_visionnage", "likes", "id_utilisateur",
"id_film", "id_episode"])

    vid = 1

    target_2017 = int(N_VISIONNAGES * PCT_2017)

    while vid <= N_VISIONNAGES:

        u = random.randint(1, N_USERS)

        if vid <= target_2017:

            d = rand_date(START_2017, END_2017)

        else:

            d = rand_date(ABO_START, ABO_END)

        if user_plan[u] == 1 and d.year == 2025 and d.month == 12 and basique_count_dec2025[u] >= 50:

            continue

        if random.random() < 0.5:

            fid = random.choice(star_films) if random.random() < 0.30 else random.randint(1, N_FILMS)

            limite = film_limit[fid]

            eid_val = NULL

            fid_val = fid

        else:

            # episode : 30% du temps un épisode venant d'une star_serie

            if random.random() < 0.30:

                # choisir un épisode d'une star serie

                star_sid = random.choice(star_series)

                eps_star = [e for e in episodes if e[1] == star_sid]

                if eps_star:

                    eid_tmp, sid = random.choice(eps_star)

                else:

                    eid_tmp, sid = random.choice(episodes)
```

```
            else:
                eid_tmp, sid = random.choice(episodes)
            limite = serie_limit[sid]
            fid_val = NULL
            eid_val = eid_tmp
        if age_on(user_birth[u], d) < limite:
            continue
        likes_base = 0.30  # 30% de like
        if (fid_val in star_films) or (fid_val == NULL and sid in star_series):
            likes_base = 0.45
        likes = 1 if random.random() < likes_base else 0
        w.writerow([vid, fmt(d), random.randint(10, 180), likes, u, fid_val, eid_val])
        if user_plan[u] == 1 and d.year == 2025 and d.month == 12:
            basique_count_dec2025[u] += 1
        vid += 1
forced_film = 1
forced_count = 130  # garantit >100
with open("telecharge.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["id_telechargement", "date_telechargement", "id_utilisateur", "id_film", "id_episode"])
    tid = 1
    while forced_count > 0:
        u = random.randint(1, N_USERS)
        if user_plan[u] != 2:
            continue
        d = rand_date(ABO_START, ABO_END)
        if age_on(user_birth[u], d) < film_limit[forced_film]:
            continue
        w.writerow([tid, fmt(d), u, forced_film, NULL])
        tid += 1
        forced_count -= 1
    while tid <= N_TELECH:
        u = random.randint(1, N_USERS)
        if user_plan[u] != 2:
            continue
        d = rand_date(ABO_START, ABO_END)
        if random.random() < 0.5:
```

```
            fid = random.randint(1, N_FILMS)
            limite = film_limit[fid]
            eid_val = NULL
            fid_val = fid
        else:
            eid_tmp, sid = random.choice(episodes)
            limite = serie_limit[sid]
            fid_val = NULL
            eid_val = eid_tmp
        if age_on(user_birth[u], d) < limite:
            continue
        w.writerow([tid, fmt(d), u, fid_val, eid_val])
        tid += 1


print("OK: CSV générés")
```